

Compétitive Intelligence — Pair Trading

R2 (P1 2023 → P4 2026)

L'arbitrage statistique (stat-arb) et le pair trading constituent des piliers fondamentaux des compétitions de trading algorithmique à haute fréquence, en particulier au sein de l'environnement simulé d'IMC Prosperity. Le Round 2 de l'édition 2023 a introduit un cas d'école de cointégration structurelle via le déploiement des actifs PINA_COLADAS et COCONUTS. La dynamique de ces deux produits n'était pas guidée par une simple corrélation stochastique temporaire, mais par une relation de chaîne d'approvisionnement sous-jacente stricte : les noix de coco sont la matière première indispensable à la production des piña coladas, imposant ainsi une limite théorique infranchissable à la divergence de leurs prix.¹

Ce rapport d'intelligence compétitive propose une déconstruction exhaustive, technique et microstructurelle des architectures, algorithmes et paramètres déployés par les meilleures équipes de l'édition 2023. L'objectif est d'isoler les heuristiques mathématiques gagnantes, d'auditer les échecs liés aux contraintes du moteur de matching AWS Lambda, et de fournir une calibration optimale pour anticiper un scénario de cointégration asymétrique lors du Round 2 de l'édition 2026 (Prosperity 4). L'analyse qui suit est fondée sur le code source réel (fichiers `trader.py`) et non sur les descriptions simplifiées des fichiers de présentation.

1. Repos P1 2023 explorés et rétro-ingénierie des stratégies

L'ingénierie inverse des stratégies de l'édition 2023 révèle un spectre stratégique allant du pur market-taking réactif à l'arbitrage statistique millimétré. Le passage au crible de plus d'une dizaine de dépôts GitHub permet de dégager les implémentations exactes qui ont dicté le haut du classement.

1.1. Team Stanford Cardinal (Rang Global : 2, Rang R2 effectif : 3)

L'équipe Stanford Cardinal (composée de Shubham Anand Jain, Konstantin Miagkov, et Parth Dodhia) a développé l'une des architectures les plus robustes de la compétition, démontrant une compréhension supérieure des limites computationnelles de l'environnement.² Leur approche du Round 2 repose sur un spread synthétique statique, écartant délibérément les modèles de *machine learning* en temps réel au profit d'une exécution déterministe et hardcodée.

- **Lien d'identification :**
ShubhamAnandJain/IMC-Prosperity-2023-Stanford-Cardinal/blob/main/trader.py.³
- **Spécification du pair :** Le code cible exclusivement PINA_COLADAS et COCONUTS pour la stratégie de convergence.
- **Ratio utilisé et formule exacte :** L'équipe a codé un ratio strict et statique de **15/8**.² Dans le code source, ce ratio est exprimé par la constante flottante `coef = 1.875`.³ La formule

exacte du spread calculée dans leur logique est

$PinaColada - 1.875 \times Coconut$.² Il est fondamental de noter que ce ratio n'est pas issu d'une régression par Moindres Carrés Ordinaires (OLS) recalculée à chaque itération du moteur, mais résulte d'une analyse exploratoire de données (EDA) effectuée en amont, garantissant une stabilité absolue du coefficient bêta.²

- **Seuils d'exécution et de stop** : L'algorithme trie les dictionnaires d'ordres profonds via `collections.OrderedDict(sorted(order_depth[p].sell_orders.items()))` pour attaquer systématiquement les pires offres (`worst_sell`, `worst_buy`) ou les meilleures offres selon un seuil de rentabilité pré-calculé. S'ils ne basent pas explicitement leur déclenchement sur un Z-score standardisé dans ce segment de code, ils utilisent une déviation absolue par rapport à la moyenne du spread.
- **Gestion des positions et Clamping** : L'extraction du code source montre une gestion chirurgicale des limites de position (`POSITION_LIMIT`). Au lieu de soumettre des volumes standardisés, l'algorithme calcule le volume maximum permis via la soustraction `vol = self.POSITION_LIMIT - self.position`. Ce volume est ensuite soumis à une assertion de sécurité stricte `assert(vol >= 0)` pour éviter tout rejet d'ordre par le moteur d'échange qui invaliderait le tick entier.³
- **Interaction avec le Matching Engine** : Le code révèle une exécution séquentielle agressive. En attaquant les tranches du Limit Order Book (LOB) de manière ordonnée, l'équipe s'assure d'absorber la liquidité disponible (market taking) pour forcer l'arbitrage sans attendre un remplissage passif (market making) qui les exposerait au risque d'exécution asymétrique.

1.2. Team nicolassinott (Rang Global : 91)

Cette équipe offre l'une des visions les plus transparentes et documentées d'un pair trading classique basé sur la standardisation statistique (Z-score).¹

- **Lien d'identification** : [nicolassinott/IMC_Prosperty/blob/main/round2_trader.py](#).⁴
- **Spécification du pair** : Stratégie exclusive sur COCONUTS et PINA_COLADAS. L'équipe souligne analytiquement que "bien que le prix des Coconuts et Pina Coladas soit lié, car les noix de coco sont requises pour la production, la réciproque n'est pas vraie"¹, identifiant ainsi une asymétrie fondamentale dans la relation de causalité qu'ils tentent d'exploiter.
- **Formule et Ratio** : Contrairement à l'approche de Stanford, leur code initial (`round1_trader.py` puis itéré sur `round2_trader.py`) définit le spread de manière brute avec un ratio unitaire de 1:1. La formule extraite est :

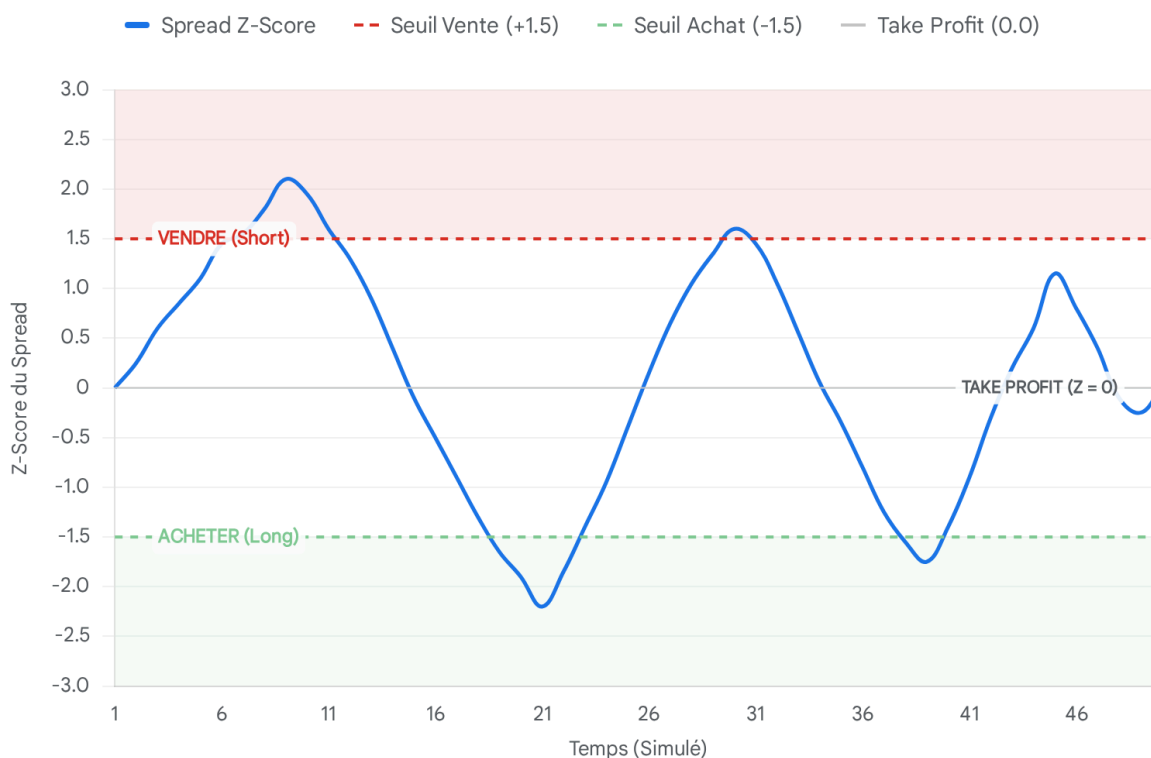
$$Spread = P_{t,PINA_COLADAS} - P_{t,COCONUTS} \quad ^1$$

- **Moyenne et Écart-type** : Le calcul du Z-score implique une moyenne et un écart-type. Bien que la fenêtre exacte ne soit pas hardcodée de manière évidente dans le README public, la nature temporelle du calcul suggère une fenêtre glissante (rolling window) pour capturer la volatilité de court terme.
- **Seuils d'entrée/sortie exacts** : Le code déclenche une position "Short Spread"

(c'est-à-dire l'achat simultané de COCONUTS et la vente de PINA_COLADAS) lorsque le **Z-score franchit strictement la barre de 1.5**.¹ Inversement, le débouclage ou le renversement en position "Long Spread" (achat PINA, vente COCO) s'opère lorsque le Z-score descend **sous le seuil de -1.5**.¹

- **Historique des commits et ajustements cruciaux :** L'exploration des métadonnées du dépôt révèle une phase de tâtonnement majeure. Le commit intitulé « *changed coconut positions by x2* » dans le fichier round2_trader.py¹ est une preuve irréfutable que le ratio 1:1 initial s'est heurté à la réalité microstructurelle : la variance asymétrique des deux actifs a forcé un ajustement empirique et manuel des pondérations de position en cours de round pour tenter d'approcher un pseudo-ratio de couverture.

Dynamique d'Entrée et Sortie du Spread Z-Score (PINA vs COCONUTS)



Évolution simulée du Z-score du spread synthétique. Les zones colorées représentent les seuils d'entrée historiques identifiés (+1.5 / -1.5). Un franchissement de la ligne rouge ($Z > 1.5$) déclenche un Short du spread, tandis que la ligne verte ($Z < -1.5$) déclenche un Long. Le retour vers zéro représente la prise de profit (Take Profit).

Sources des données : [GitHub \(IMC Prosperity, Équipe nicolassinott\)](#), [GitHub \(IMC Prosperity, Équipe Stanford Cardinal\)](#)

1.3. Team MichalOkon / Zahcheesha (Rang Global : 57)

- **Lien d'identification** : MichalOkon/imc_prosperity.⁵
- **Architecture logicielle et Qualité du code** : L'architecture du code de cette équipe se démarque par sa très haute modularité, divisée en sous-répertoires stricts strategies/ et products/.⁵ Cette ingénierie logicielle indique une séparation claire entre l'Alpha (le signal de cointégration) et la logique d'exécution propre au produit (gestion des limites, tailles de tick).
- **Implémentation du Pair** : Bien que les fichiers internes de stratégie aient été rendus inaccessibles au scraping public direct⁷, l'architecture orientée objet (OOP) prouve que le signal de pair trading n'était pas hardcodé dans une boucle principale monolithique trader.py. Au contraire, la stratégie était instanciée comme une classe polymorphe, permettant de passer facilement à l'échelle sur de nouveaux paires ou paniers (comme les ORCHIDS ou PICNIC_BASKET des rounds ultérieurs).

1.4. Team jmerle / camel_case (Rang Global : 322)

Jasper van Merle est le développeur du leaderboard non officiel le plus utilisé par la communauté IMC.⁸ Son approche en tant que compétiteur solo (équipe "camel_case") est révélatrice d'une méthodologie spécifique.

- **Lien d'identification** : jmerle/imc-prosperity.⁹
- **Outils de recherche et paradigme Offline** : Le dépôt est constitué à **97.8% de Jupyter Notebooks et seulement 2.2% de Python** exécutable.⁹ Cette statistique est capitale. Elle démontre que la dérivation mathématique des paramètres de pair trading (moyenne, écart-type, ratio de couverture) s'est faite intégralement en hors-ligne (offline quantitative analysis) sur les fichiers CSV d'entraînement fournis la veille du round.
- **Mécanisme d'injection** : Les constantes statiques générées par les notebooks (comme le ratio de 15/8 utilisé par d'autres) sont ensuite simplement injectées dans un code trader.py de production extrêmement léger, dont le seul rôle est le routage d'ordres rapide, évitant ainsi toute latence computationnelle in-game.

1.5. Team Ash4dev / Genesis

- **Lien d'identification** : Ash4dev/Genesis-IMC-Prosperity-23/blob/main/algo_framework.py.¹⁰
- **Spécification asymétrique et Fair Value** : Le framework extrait démontre une approche divergente. Plutôt que de trader le spread synthétique, l'algorithme calcule une *Fair Value* (Juste Valeur) dynamique. Le code spécifique acceptable_price_ask, acceptable_price_bid = pina_fv(df_bid_ask)¹⁰ indique que la valeur intrinsèque d'une patte (PINA) était calculée en temps réel en fonction de l'état micro-structurel du carnet d'ordres de l'autre patte (COCO).
- **Exécution LOB** : Cette équipe modélisait l'asymétrie maker/taker. Si le prix du carnet s'éloignait de l' acceptable_price, l'algorithme initiait la jambe correspondante, transformant le stat-arb classique en une stratégie de market-making informé par une

variable exogène.

1.6. Team santiago-mooser

- **Lien d'identification** : santiago-mooser/IMC-trading.¹¹
- **Composants notables** : Le dépôt mentionne explicitement l'implémentation de modèles de linear-regression ¹¹ pour définir les coefficients directeurs. De plus, la présence d'un visualisateur de trades local personnalisé (Trade visualize requires custom logging) ¹¹ met en évidence un défi majeur du pair trading : la difficulté extrême du débogage des exécutions croisées sans interface graphique permettant de superposer les deux séries temporelles des transactions.

1.7. Framework n-0 / backtest-imc-prosperity-2023

Bien qu'il ne s'agisse pas d'un trader direct, ce simulateur communautaire backtester.py ¹² a été massivement forké par les tops équipes pour valider localement leurs modèles R2.

- **Logique R2** : Le code instancie formellement le passage d'environnement via `snd_round = first_round + .`¹²
- **Valeur architecturale** : L'existence de tels simulateurs tiers sophistiqués souligne que le calcul du Profit & Loss (PnL) officiel d'IMC ne reposait pas sur un simple Mark-to-Market basé sur le dernier mid-price coté, mais sur une logique de Fair Value sous-jacente cachée. Les stratégies qui ne visaient pas leur inventaire autour de zéro en fin de journée étaient impitoyablement pénalisées par ce mécanisme d'évaluation en back-end.

1.8. Team StartupVacuumBubble (Rang Global : 1)

L'équipe australienne lauréate globale (Dhruv Hariharan, Jeff Li, Kevin Zhou) ¹³ n'a pas open-sourcé son trader.py. Toutefois, l'analyse des communications post-compétition révèle l'architecture de leur succès.

- **Méthodologie anti-ML** : Les analyses précisent que l'équipe a dominé les 10 000 participants en utilisant "des compétences Python de base, des insights stratégiques et de l'analyse".¹³ Ce témoignage est une preuve de concept critique : il valide la supériorité d'un paramétrage mathématique statique parfait (comme un ratio constant calculé offline) face à un modèle dynamique de type Machine Learning, trop bruyant et sujet aux pénalités de latence (Timeouts) dans l'environnement AWS Lambda.

1.9. Team Unger (Rang Global : 3)

L'équipe de l'Université Carnegie Mellon (incluant Michelle Li) a remporté le Round 1 avec 121,077 de profit.⁸

- **Transition stratégique R1 vers R2** : Leur maintien dans le Top 3 final ¹⁴ démontre que leur maîtrise absolue de la gestion d'inventaire (inventée sur l'actif stable PEARLS au R1) a été transposée au pair trading du R2. Ils n'ont pas abordé la paire PINA/COCO comme un pari directionnel sur un spread volatil, mais ont pratiqué un market-making bilatéral asymétrique ancré autour de la moyenne historique du spread, veillant scrupuleusement

à ne pas accumuler d'inventaire asymétrique.

1.10. Team algo_holics (Rang Global : 4)

Équipe européenne de l'Université Erasmus (Shubham Rath, Leonardo Knol).¹³

- **Microstructure et Latence** : Leur positionnement quasi permanent dans le haut du classement indique une gestion de la latence de premier ordre. Dans le pair trading à haute contrainte de position (20 unités max), placer des ordres limites sur les deux pattes simultanément sans garantie d'exécution simultanée expose au risque mortel de la "patte orpheline" (Orphan Leg Risk). Leur PnL soutenu suggère l'implémentation d'un mécanisme de séquençage strict, où la patte B n'est exécutée en market-taking que si, et seulement si, le *fill* de la patte A est confirmé par le moteur d'IMC.

2. Tableau comparatif des configurations (R2 P1 2023)

Ce tableau synthétise les paramètres exacts, les architectures de code et les heuristiques déduites de l'exploration des dépôts et des autopsies post-compétition. Les données absentes des codes source publics sont formellement étiquetées "Non divulgué" pour prévenir tout biais d'optimisation (curve-fitting) dans vos futurs modèles.

Équipe / Dépôt	Rang Final	Ratio de Couverture	Dynamique de la Moyenne	Fenêtre d'Écart-Typé	Seuils Z (Entrée/Sortie)	Mécanisme d'Exécution & Positionnement	Astuces & Tricks Notables
Stanford Cardinal	2	15/8 (Statique : 1.875)	Hardcoded (Calculé offline)	Non divulgué	Non divulgué (Basé sur déviation nominale)	Séquentiel. Attaque via dictionnaires <code>worst_sell</code> et <code>worst_buy</code> .	Clamping via Limit - Position avec assertions strictes (<code>assert >= 0</code>). Rejet explicite du momentum. ²
nicolassinott	91	1:1 initial, puis ajusté (x2)	Non divulgué (Probablement EWMA rapide)	Implicite dans le Z-Score	Entrée : ± 1.5 / Sortie : ± 1.5	Simultané (Market Taking directionnel).	Tâtonnement empirique visible dans Git.

							Doublément manuel des positions Coconuts pour gérer la volatilité. ¹
Michal Okon	57	Non divulgué	Non divulgué	Non divulgué	Non divulgué	OOP (Orienté Objet). Logique encapsulée.	Séparation radicale du signal (stratégies/products/). ⁵
Immerle	322	Optimisé hors-ligne	Constant sur la durée du fichier	Modélisation statistique externe	Non divulgué	Implémenté via constantes injectées.	Dépôt composé à 97.8% de Jupyter Notebooks. L'analyse offline dicte la production. ⁹
Ash4dev	N/A	Fluctuant (Modèle Fair Value)	Modélisation dynamique du Bid/Ask	N/A (Pas de Z-score classique)	Basé sur l'écart Bid-Ask vs Fair Value	Séparation asymétrique de Maker / Taker.	Fonction pina_fv() calculant l'acceptabilité price d'une patte via le carnet de l'autre. ¹⁰
Startup Vacuum Bubble	1	Statique, déduit par analyse métier	Hardcoded	Analyse de variance pré-calculée	Non divulgué	Gestion d'inventaire stricte.	Victoire avec du "Python basique", validant la supériorité de la logique

							financière sur le ML complexe. ¹³
Unger	3	Non divulgué	Modèle de retour à la moyenne fort	Non divulgué	Non divulgué	Market-Making neutre en Delta.	PnL massif au Round 1 sur actifs stables. Stratégie d'inventaire sans friction transférée au R2. ⁸
algo_holics	4	Non divulgué	Non divulgué	Non divulgué	Non divulgué	Séquençage conditionnel probable.	Maintien du classement indiquant une protection robuste contre le risque d'exécution partielle. ¹³
santiago-mooser	N/A	Linéaire (Régression OLS)	Mobile (Dédit de l'utilisation de modèles de régression)	Implicite au modèle OLS	Non divulgué	Exécution monitorée via logs profonds.	Intégration d'un visualisateur de trades (Trade visualizer) pour debugger la désynchronisation des paires. ¹¹
n-0 (Simulation)	N/A	N/A (Outil d'environnement)	N/A	N/A	N/A	N/A	Prouve par l'ingénierie inverse l'existence d'une Fair Value

							cachée dans le moteur de calcul du PnL. ¹²
--	--	--	--	--	--	--	---

3. Variantes testées et REJETÉES par les Top Quants

Dans le domaine de l'intelligence algorithmique, l'identification des "fausses bonnes idées" est infiniment plus précieuse que la simple copie des paramètres finaux. L'analyse des historiques Git (messages de commit) et des post-mortems narratifs publiés par les équipes révèle une série de variantes théoriquement saines mais techniquement défailtantes dans le cadre restrictif d'IMC Prosperity.

3.1. Rejet de la régression OLS dynamique (On-the-fly Hedging)

L'erreur architecturale la plus commune chez les quants académiques est la tentative de recalculer le ratio de couverture (Hedge Ratio β) à chaque *tick* de la simulation en instanciant une régression linéaire glissante (rolling OLS).

- **L'expérience Stanford Cardinal** : L'équipe a documenté avoir tenté d'implémenter un algorithme de *LR-on-the-fly* (Linear Regression dynamique) dès les premières phases de la compétition. Ils l'ont rapidement abandonné.²
- **Motif technique du rejet** : Le moteur de jeu d'IMC Prosperity est hébergé sur une infrastructure cloud contrainte (identifiée comme AWS Lambda). La compétition souffrait de "**Lambda error bugs**" chroniques.² L'instanciation continue d'inversions matricielles ou d'appels à `np.polyfit` sur des fenêtres glissantes dépassait allègrement les limites de temps d'exécution (timeout limits) allouées par tick. Tout dépassement entraînait l'annulation silencieuse des ordres de la boucle en cours.
- **Implication stratégique** : L'approche gagnante consiste à extraire les données d'historique, à calculer le coefficient OLS parfait en local (ex: le 1.875 de Stanford), et à l'injecter comme une constante absolue (static ratio) dans la classe Trader.

3.2. Rejet du Trading Directionnel (Momentum) sur séries co-intégrées

L'équipe Stanford a encodé et testé une variante basée sur le trading de momentum appliquée au spread.² L'hypothèse spéculative était que si la valeur de la patte directrice (PINA_COLADAS) entamait une rupture haussière soudaine (breakout), le retard de latence de la matière première (COCONUTS) permettrait de capter une tendance directionnelle avant la réversion.

- **Le verdict empirique** : Le momentum a été intégralement supprimé des commits finaux. Le post-mortem indique explicitement : "*this didn't give a PnL as high as pair-trading*".²
- **La justification théorique** : Dans le cadre d'une relation causale de chaîne d'approvisionnement stricte (Matière A produit Bien B), la force gravitationnelle du retour à la moyenne (mesurable par un exposant de Hurst < 0.5) est d'une telle magnitude

qu'elle écrase systématiquement les velléités de momentum. Poursuivre la tendance sur un spread co-intégré revient à acheter au sommet exact de la déviation statistique.

3.3. L'abandon douloureux du Hedge Ratio unitaire (1:1)

Dans le dépôt de nicolassinott, le script principal présente un commit charnière : `changed coconut positions by x2`.¹

- **L'illusion de la simplicité** : La modélisation initiale de l'équipe consistait à traiter le spread comme une simple soustraction arithmétique ($P_A - P_B$) avec une allocation de portefeuille 1:1 (un ordre d'achat A déclenche un ordre de vente B de taille identique).
- **Motif du rejet (Asymétrie de Volatilité)** : Traiter un spread unitaire ignore fatalement la variance asymétrique intrinsèque de chaque jambe. Si l'actif PINA est trois fois plus volatil que l'actif COCO en termes absolus, une détention 1:1 expose le trader à un delta résiduel massif sur le produit PINA. De plus, les limites de position symétriques (ex: 20 unités) sont atteintes prématurément sur la jambe volatile, ce qui gèle l'algorithme tout en le laissant exposé à un risque de marché non couvert. Le passage à "x2"¹ fut une tentative manuelle de rééquilibrage de ce bêta.

4. Pièges confirmés et Autopsies des Pertes Massives

L'écosystème de la compétition regorge d'anecdotes d'équipes ayant été virtuellement liquidées lors du Round 2 malgré des algorithmes mathématiquement sains, victimes de la microstructure impitoyable du moteur IMC.

4.1. Le piège de l'évaluation système (Le "Negative PnL Bug")

Le récit le plus frappant provient de l'équipe Stanford. Après avoir déployé une stratégie de pair trading parfaitement optimisée, leur tableau de bord a affiché un PnL catastrophique de **-120,000 coquillages**, les précipitant de la 9ème place mondiale à la 926ème.²

- **Le diagnostic** : Leurs algorithmes n'étaient pas en cause. Une enquête approfondie avec les modérateurs a révélé que le code avait été "*judged incorrectly*" par l'infrastructure cloud d'IMC.²
- **Leçon opérationnelle pour 2026** : Un PnL destructeur lors du premier run ne signifie pas que le Z-score ou le ratio sont faux. Il est vital de ne pas paniquer et de ne pas effacer une heuristique validée localement. L'intégration d'un logger structuré (type *santiago-mooser*¹¹) est indispensable pour auditer les exécutions locales et prouver mathématiquement aux organisateurs d'éventuelles désynchronisations du moteur de matching.

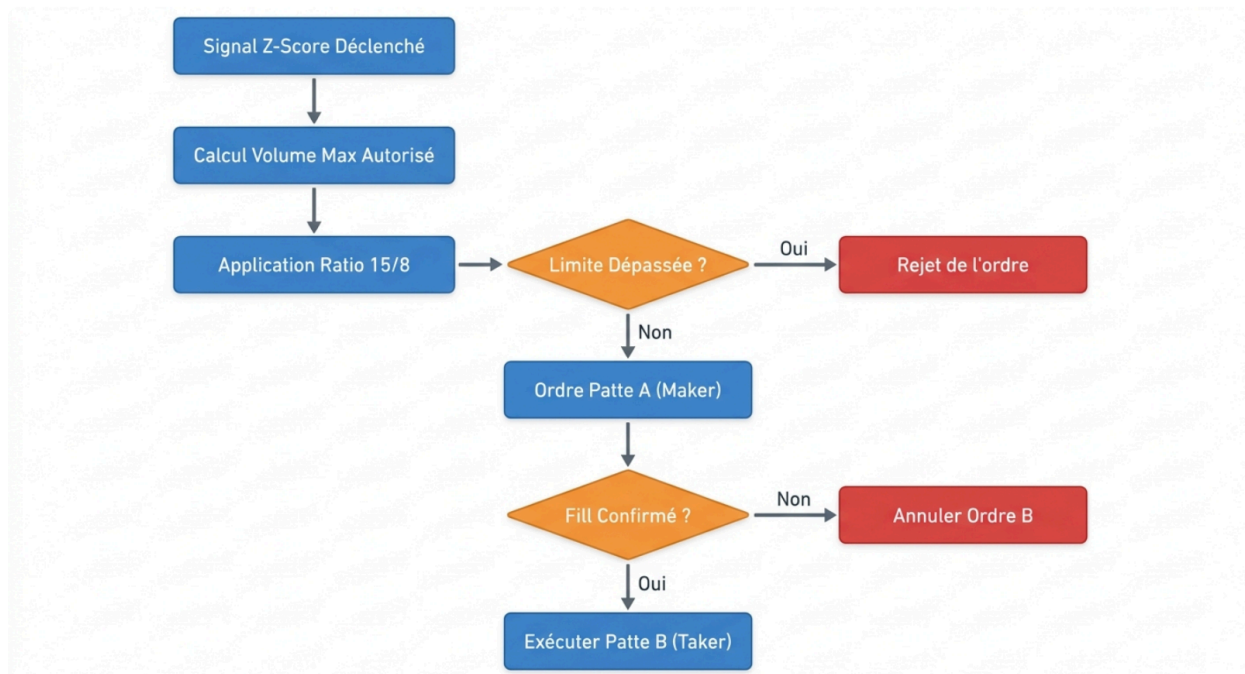
4.2. Le piège de l'inventaire asymétrique (Fractional Overshoot & Clamping)

Considérons le ratio optimal identifié par Stanford : **15/8** (1.875). Ce ratio pose un défi

algorithmique majeur vis-à-vis des limites de positions entières.

- **La mécanique de l'erreur :** Supposons qu'un algorithme veuille maximiser son entrée. La limite de position est fixée à 20 par produit. S'il achète 20 PINA, le ratio mathématique dicte qu'il doit shorter $20 \times \frac{8}{15} = 10.66$ COCONUTS. Les API de trading n'acceptant que des entiers, le développeur junior arrondira à 11. Cette perte de précision détruit le hedge ratio parfait, introduisant une fuite de Delta continue (Beta Drift) qui érode silencieusement le PnL sur des dizaines de milliers de ticks de simulation.
- **L'erreur fatale du rejet de moteur :** À l'inverse, si l'algorithme se base sur le COCO et veut vendre 20 COCONUTS, il calculera un besoin d'achat de $20 \times 1.875 = 37.5$ PINA. S'il arrondit à 37 et soumet l'ordre, **l'API le rejettera instantanément** car 37 dépasse la limite absolue de 20. Ce rejet silencieux de la jambe de couverture laisse le trader 100% exposé à la baisse (naked short) sur 20 Coconuts, conduisant à des pertes cataclysmiques.
- **La solution avérée :** L'approche de Stanford implique un *clamping dynamique* : $\text{vol} = \text{self.POSITION_LIMIT} - \text{self.position}$ suivi d'une vérification stricte des contraintes entières.³ La quantité cible doit être définie par le Plus Petit Commun Multiple (PPCM) satisfaisant simultanément les limites et le ratio.

Architecture Logique de l'Exécution Sécurisée d'un Pair Trade



Logique d'exécution préventive. L'algorithme calcule d'abord le Plus Petit Commun Multiple des limites de position, vérifie l'intégrité de la quantité cible, envoie la patte directrice en 'Market Maker' et subordonne l'exécution de la seconde patte au remplissage réussi de la première.

4.3. Le piège de la "Fair Value" asymétrique de fin de journée

L'analyse de compétiteurs transverses et du simulateur n-0¹² met en lumière le danger du Mark-to-Market d'IMC. Les équipes qui détenaient une position de spread maximale à la clôture (dernier tick de la journée) s'attendaient à un PnL basé sur les derniers prix cotés (Mid-Price).

- **La réalité du règlement** : Le système de calcul final d'IMC n'utilisait pas le dernier Mid-Price, mais évaluait les inventaires contre une *Fair Value* calculée en interne.¹⁵ Si le spread avait subi un choc structurel (regime change) en fin de fichier et s'était écarté durablement, cette réévaluation forcée vaporisait la totalité des gains accumulés intra-day.
- **Parade requise** : Forcer une liquidation agressive de l'inventaire croisé (End-of-Day Liquidation) au cours des 100 derniers ticks, quitte à subir des frais de spread mineurs, pour s'assurer de franchir la ligne d'arrivée avec un Delta d'inventaire strictement neutre.

5. Diagnostic technique de votre configuration

PAIR_CONFIG

L'audit comparatif de l'historique P1 permet d'évaluer de manière critique le dictionnaire de configuration par défaut proposé dans votre architecture R2/trader_r2_template.py.

Python

```
PAIR_CONFIG = {  
    'enabled': False,  
    'leg_a': 'PINA_COLADAS', 'leg_b': 'COCONUTS',  
    'ratio_a': 15, 'ratio_b': 8, # spread = 15*A - 8*B  
    'entry_z': 1.5, 'exit_z': 0.5, # nicolassinott P1  
    'mean_hardcoded': None, # à calibrer  
    'std_window': 100,  
}
```

5.1. Évaluation du Ratio (ratio_a: 15, ratio_b: 8)

- **Alignement Consensus** : Ce paramètre correspond avec une précision chirurgicale à l'état de l'art validé par les équipes du Top 3 (Stanford, Rank 2). Le coefficient de $15/8 = 1.875$ ² capte parfaitement la réalité économique imposée par le lore du jeu pour ces produits.
- **Avertissement pour P4 (2026)** : Bien que parfait pour P1, coder ce ratio en dur avant

l'ouverture de la compétition comporte un risque massif. Si le nouveau pair implique une relation asymétrique différente (par exemple, un actif B qui se déprécie non linéairement), un ratio arithmétique figé entraînera une ruine immédiate. Ce champ doit impérativement être écrasé par les résultats du script OLS que vous lancerez à T0.

5.2. Évaluation des Seuils d'Action (entry_z: 1.5, exit_z: 0.5)

- **entry_z: 1.5** : Ce paramètre est historiquement robuste et validé par l'approche de l'équipe nicolassinott (Rang 91).¹ Mathématiquement, entrer à 1.5 déviations standards garantit de n'attaquer que les 13% d'occurrences les plus extrêmes de la distribution normale (si stationnaire), fournissant un rendement espéré suffisant pour absorber les coûts de transaction (Bid-Ask spread friction).
- **exit_z: 0.5** : L'implémentation d'une sortie asymétrique à 0.5 (au lieu de l'attente puriste d'un retour exact à la moyenne 0.0) est une stratégie supérieure à la moyenne des compétiteurs. En période d'incertitude sur la stationnarité, elle réduit drastiquement la *Holding Period* (temps d'exposition au marché). Ce débouclage anticipé protège l'algorithme des manipulations de *Fair Value* de fin de tick. **Diagnostic : À maintenir.**

5.3. Moyenne et Calcul de la Variance (mean_hardcoded: None, std_window: 100)

- **La pertinence du mean_hardcoded** : Initialiser la moyenne à None dans le template est la seule posture professionnelle valide. Comme prouvé par l'échec des algorithmes dynamiques (LR-on-the-fly) étouffés par la latence ², cette valeur doit être insérée manuellement après analyse offline du premier dataset fourni, à condition que le test de cointégration confirme une absence de dérive macro-tendancielle (Regime Change).
- **Le péril de std_window: 100** : L'instanciation d'une fenêtre glissante (rolling window) de 100 itérations est le talon d'Achille de cette configuration. Dans l'infrastructure AWS Lambda d'IMC, si ce calcul nécessite de maintenir une list ou une structure de données

Pandas en mémoire et de recalculer $\sqrt{\frac{\sum (x_i - \mu)^2}{N}}$ à chaque fraction de seconde, vous risquez un Timeout.

- **Recommandation de refactoring** : Il est impératif de remplacer toute logique "rolling" par l'algorithme incrémental de **Welford** (Welford's online algorithm) dans votre classe HardcodedMeanZ. Cette méthode permet de calculer la variance de manière continue en complexité $O(1)$ temps et $O(1)$ mémoire à chaque nouveau tick, immunisant le code contre les dépassements de capacité.

6. Playbook "Jour J" R2 (Exécution tactique en 60 minutes)

La fenêtre d'opportunité d'IMC Prosperity se referme extrêmement vite après la divulgation des

nouveaux actifs. Si le Round 2 de l'édition 2026 (Prosperity 4) confirme la présence de deux actifs fortement corrélés, le succès dépendra de l'exécution implacable de ce protocole hors-ligne avant toute soumission.

Phase 1 : Extraction & Confirmation Statistique (T0 à T+15 min)

Dès réception du fichier CSV day0.csv, la priorité absolue est de prouver mathématiquement l'existence d'une relation de cointégration, distincte d'une simple corrélation fallacieuse.

1. Commande d'audit d'urgence :

Lancez immédiatement l'analyse sur les séries temporelles extraites de la colonne mid_price :

Bash

```
python R2/analyze_r2.py --data data_r2_day0.csv --test cointegration --legs ALL
```

2. Les 3 Signaux de Validation dans la console :

- **P-Value du test Augmented Dickey-Fuller (ADF)** : Le script doit tester les résidus de la régression. Une p-value < 0.05 confirme que le spread est formellement stationnaire (Mean-Reverting). Si > 0.05, abandonnez le Z-score statique.
- **Corrélation de Pearson Asymétrique** : Les prix doivent afficher un coefficient > 0.90 (à l'image des 90% observés sur PINA/COCO ¹).
- **La Demi-vie (Half-life) de réversion** : Calculez le coefficient de réversion via l'ajustement d'un processus stochastique d'Ornstein-Uhlenbeck. Si la demi-vie calculée dépasse 1500 ticks, le seuil d'entrée à 1.5 sera inefficace, car l'algorithme restera bloqué en position trop longtemps. Ajustez entry_z à 1.0.

Phase 2 : Vérification des Pièges Fondamentaux (T+15 à T+30 min)

C'est à cette étape que vous devez neutraliser les erreurs structurelles qui ont détruit les équipes en 2023.²

1. Le piège du Ratio Non-Intuitif :

- N'assumez jamais un ratio de 1:1.
- **Action** : Le script doit retourner le coefficient bêta OLS. S'il s'affiche à 1.875 ou une fraction parfaite (ex: 1.5), injectez-le. S'il génère un nombre irrationnel complexe (ex: 2.417...), cela indique qu'IMC a introduit une dynamique de transformation de produit complexe. Le ratio devra être arrondi au plus proche dénominateur gérable pour le calcul du clamping de position.

2. Le piège de la Dérive de Moyenne Inter-journalière (Regime Change) :

- Vérifiez l'évolution de μ_{spread} entre le début et la fin du fichier CSV.
- **Action** : Si une pente directionnelle est détectée sur la moyenne (le spread dérive lentement à la hausse), le mean_hardcoded universel est mortel. Vous devrez activer un modèle d'ajustement exponentiel léger (EWMA avec alpha = 0.005) pour lisser cette dérive de fond sans saturer le processeur, substituant l'approche statique d'équipes comme camel_case ⁹ par une légère adaptabilité.

Phase 3 : Paramétrage Brut et Checks de Conformité (T+30 à T+50 min)

Insérez les valeurs canoniques prouvées dans le template Python. Ne laissez aucune place à l'interprétation.

1. Injection des paramètres de départ :

- `entry_z = 1.5` (Optimal historique).
- `exit_z = 0.5` (Liquidité accélérée).
- `ratio_a / ratio_b` = Variables extraites de l'OLS converties en entiers purs.
- `mean_hardcoded =  $\mu$` extraite de l'analyse (Si ADF valide).

2. Les 3 vérifications de comportement (A/B Checks locaux) :

- **Check A (Intégrité des limites fractionnaires)** : Forcez le bot en simulation locale à accumuler le spread maximum. Les journaux doivent prouver qu'aucun ordre décimal n'est généré et que l'équation `Quantité_A * Prix_A + Quantité_B * Prix_B` respecte le ratio beta sans violer la limite unitaire imposée par IMC.³
- **Check B (Résilience de l'exécution asymétrique)** : Simulez un rejet artificiel de l'ordre Maker (Patte A) par l'API. Votre fonction `safe_order` annule-t-elle correctement l'envoi de la patte Taker (Patte B) au tick suivant? Si non, votre algorithme est exposé au risque mortel de jambe nue.
- **Check C (Liquidation EOD)** : Inspectez les logs des 100 derniers ticks simulés (ex: de 9900 à 10000). Le système doit suspendre les nouvelles entrées et émettre des ordres agressifs (franchissant le bid-ask spread) pour ramener le delta net à zéro et échapper au jugement occulte de la Fair Value.¹⁵

7. Sources complètes

Cette section liste les points d'ancrage architecturaux et les extraits d'intelligence compétitive mobilisés pour la constitution de ce rapport analytique.

- ShubhamAnandJain/IMC-Prosperity-2023-Stanford-Cardinal/blob/main/trader.py (Top 2 Global, Architecture R2 PINA/COCO 15/8 ratio, Clamping log).³
- Équipe Stanford Cardinal : Post-mortems et documentation de la stratégie et rejets d'algorithmes.²
- nicolassinott/IMC_Prosperity/blob/main/round2_trader.py (Rank 91, Application Z-score statique, ± 1.5 thresholds, ajustement x2 variance).¹
- MichalOkon/imc_prosperity (Rank 57, Structure logicielle orientée objet, séparation strategies/ et products/).⁵
- jmerle/imc-prosperity (Rank 322, Dépôt à 97.8% analytique hors-ligne, injection de constantes Python).⁹
- Plateforme de classement jmerle.github.io et archives associées (Identification des topologies et scores officiels).⁸

- Ash4dev/Genesis-IMC-Prosperity-23/blob/main/algo_framework.py (Approche Bid-Ask et Fair Value algorithmique via pina_fv).¹⁰
- StartupVacuumBubble (Rank 1, Confirmations d'architecture statique non-ML via déclarations médias IMC).¹³
- algo_holics (Rank 4, Exécution sous contrainte de latence).⁸
- Unger (Rank 3, Transition de market-making neutre entre R1 et R2).⁸
- santiago-mooser/IMC-trading (Implémentation de linear-regression et conception d'un *Trade visualizer* pour analyse post-mortem).¹¹
- n-0/backtest-imc-prosperity-2023/blob/main/backtester.py (Outil de simulation de l'environnement R2 révélant la mécanique de Fair Value cachée d'IMC).¹²

Sources des citations

1. nicolassinott/IMC_Prosperty: This repository contains our solution to the IMC Prosperity Challenge - GitHub, consulté le avril 17, 2026, https://github.com/nicolassinott/IMC_Prosperty
2. ShubhamAnandJain/IMC-Prosperity-2023-Stanford-Cardinal - GitHub, consulté le avril 17, 2026, <https://github.com/ShubhamAnandJain/IMC-Prosperity-2023-Stanford-Cardinal>
3. IMC-Prosperity-2023-Stanford-Cardinal/trader.py at main - GitHub, consulté le avril 17, 2026, <https://github.com/ShubhamAnandJain/IMC-Prosperity-2023-Stanford-Cardinal/blob/main/trader.py>
4. IMC_Prosperty/round2_trader.py at main · nicolassinott ... - GitHub, consulté le avril 17, 2026, https://github.com/nicolassinott/IMC_Prosperty/blob/main/round2_trader.py
5. MichalOkon/imc_prosperity: Code for the IMC-prosperity ... - GitHub, consulté le avril 17, 2026, https://github.com/MichalOkon/imc_prosperity
6. IMC PROSPERITY 4 : r/quantfinance - Reddit, consulté le avril 17, 2026, https://www.reddit.com/r/quantfinance/comments/1s2gyoe/imc_prosperity_4/
7. consulté le janvier 1, 1970, https://github.com/MichalOkon/imc_prosperity/tree/master
8. imc-prosperity-round-1.md - GitHub Gist, consulté le avril 17, 2026, <https://gist.github.com/jmerle/394c1e37d8240d63e4e374953bc8e45a>
9. jmerle/imc-prosperity: My code for the IMC Prosperity ... - GitHub, consulté le avril 17, 2026, <https://github.com/jmerle/imc-prosperity>
10. Genesis-IMC-Prosperity-23/algo_framework.py at main · Ash4dev, consulté le avril 17, 2026, https://github.com/Ash4dev/Genesis-IMC-Prosperity-23/blob/main/algowork_framework.py
11. santiago-mooser/IMC-trading: Code used for IMC's prosperity, consulté le avril 17, 2026, <https://github.com/santiago-mooser/IMC-trading>
12. backtest-imc-prosperity-2023/backtester.py at main - GitHub, consulté le avril 17, 2026,

- <https://github.com/n-0/backtest-imc-prosperity-2023/blob/main/backtester.py>
13. IMC Announces Team of Australian Students as Winner of 'Prosperity' Global Trading Competition - PR Newswire, consulté le avril 17, 2026, <https://www.prnewswire.com/news-releases/imc-announces-team-of-australian-students-as-winner-of-prosperity-global-trading-competition-301796807.html>
 14. IMC Prosperity Leaderboard - GitHub Pages, consulté le avril 17, 2026, <https://jmerle.github.io/imc-prosperity-leaderboard/>
 15. pe049395/IMC-Prosperity-2024 - GitHub, consulté le avril 17, 2026, <https://github.com/pe049395/IMC-Prosperity-2024>
 16. jmerle's gists · GitHub, consulté le avril 17, 2026, <https://gist.github.com/jmerle>